

Setting	Value
Target modules	q, k, v, o, gate, up, down proj
Rank (r)	64
Scaling factor (α)	16
LoRA dropout	0.05
Bias	none
Base precision	4-bit NF4
Compute dtype	fp16

Table 32: LoRA configuration for direct preference optimization.

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} \left[\log \sigma \left(\beta (\Delta_{\theta}(x) - \Delta_{\text{ref}}(x)) \right) \right] \quad (4)$$

where

$$\begin{aligned} \Delta_{\theta}(x) &= \log \pi_{\theta}(y^+ | x) - \log \pi_{\theta}(y^- | x), \\ \Delta_{\text{ref}}(x) &= \log \pi_{\text{ref}}(y^+ | x) - \log \pi_{\text{ref}}(y^- | x), \end{aligned} \quad (5)$$

and β controls the strength of implicit KL regularization. Intuitively, DPO increases the policy’s relative log-likelihood of the preferred completion compared to the rejected one, while anchoring updates to the reference policy to prevent drift.

Data formatting for DPO Each example is converted into the TRL-style DPO schema:

- **prompt:** the fixed instruction + chatlog + question + answer options + “Return exactly one token: 1 or 2.”
- **chosen:** the preferred completion (“1” or “2”)
- **rejected:** the dispreferred completion (the opposite token)

Since the completion is one token, we append EOS and ensure the tokenizer does not introduce extra whitespace tokens. This keeps logprob accounting consistent between training and inference.

Model and parameter-efficient tuning We apply DPO using parameter-efficient fine-tuning (PEFT/LoRA) on top of Qwen2.5-7B-Instruct. The reference model π_{ref} is a frozen copy of the initial policy. Only LoRA adapter parameters are updated. Table 32 lists the configuration.

Training procedure We implement training using TRL’s DPOTrainer. The trainer optimizes the DPO objective by comparing the policy π_{θ} against the frozen reference model, with temperature parameter β controlling implicit KL regularization.

Hyperparameter	Value
Base model	Qwen2.5-7B-Instruct
Reference model π_{ref}	Frozen copy of base
β (DPO temperature)	0.1
Learning rate	5×10^{-5}
Optimizer	AdamW
LR schedule	cosine
Warmup ratio	0.03
Number of epochs	1
Per-device batch size	4
Gradient accumulation steps	4
Effective batch size	16
Max prompt length	768
Max completion length	1 token (+ EOS)
Precision	fp16

Table 33: Direct preference optimization hyperparameters.

Training uses mixed precision and gradient accumulation. We use a small per-device batch size, accumulate gradients for multiple steps, and apply optimizer updates only after accumulation, which stabilizes learning in the discrete output space. Optimization uses AdamW with a cosine learning-rate schedule and warmup. We apply gradient clipping to prevent spikes during preference updates. Table 33 summarises the hyperparameters.

Compute and software configuration All experiments use the Hugging Face Transformers and TRL libraries, with PEFT for adapter-based training. Fine-tuning is performed on a single NVIDIA A100 GPU with 80GB of memory. The base model weights are loaded in 4-bit NF4 quantization, with computation in fp16 precision. We enable gradient checkpointing to reduce activation memory. Training runs for one epoch and completes in approximately eleven hours.

Results See Table 19 for mitigation effectiveness across benchmarks.

J LLM Usage

Other than being used as part of the experiments conducted in this work, LLMs were used as both a coding and writing assistance tool in preparing this paper submission. For coding, their role was limited to debugging, syntax suggestions, and implementing routine functionality based on specifications provided by the authors. For writing, LLMs assisted with polishing language, improving clarity, and reducing redundancy, using prompts similar to “Please revise the writing of this, making sure to

remove any grammatical mistakes." All research ideas, experimental designs, algorithmic innovations, analyses, and claims presented in the paper are entirely the original work of the authors. No part of the conceptual, methodological, or empirical contributions relies on or originates from LLM outputs.